

Міністерство освіти і науки України
Харківський національний університет радіоелектроніки

Кафедра «Програмна інженерія»
Звіт з лабораторної роботи №4
З дисципліни "Моделювання та аналітика даних"
LASSO-регресія через координатний спуск

Виконала:
студентка гр. ПЗМ-25-2
Тимощенко Владислава Русланівна

Перевірів: асистент кафедри ПІ
Дашенков Дмитро Сергійович

Харків 2026

1. Мета: ознайомитися з принципами LASSO-регресії та реалізувати алгоритм координатного спуску за допомогою NumPy для відбору найбільш важливих ознак і побудови моделі прогнозування вартості будинків.

2. Хід роботи

2.1 Нормалізація ознак

Реалізуємо `normalize_features(feature_matrix)`, яка ділить кожен стовпець на його норму L2 та повертає (`normalised_features`, `norms`).

Завдання 1 — Нормалізація ознак

Реалізуйте `normalize_features(feature_matrix)`, яка ділить кожен стовпець на його норму L2 та повертає (`normalised_features`, `norms`).

```
def normalize_features(feature_matrix):  
    """  
    Ділить кожен стовпець на його норму L2.  
  
    Повертає  
    -----  
    normalised : np.ndarray, та сама форма що й feature_matrix  
    norms      : np.ndarray, shape (n_cols,)   
    """  
    norms = np.linalg.norm(feature_matrix, axis=0)  
    normalised = feature_matrix / norms  
  
    return normalised, norms
```

Перевірка — кожен стовпець `normalised` має дорівнювати [0.6, 0.8]; `norms` = [5, 10, 15]

```
test_matrix = np.array([[3., 6., 9.], [4., 8., 12.]])  
normalised, norms = normalize_features(test_matrix)  
print('Нормалізована матриця:\n', normalised)  
print('Норми:', norms)
```

```
... Нормалізована матриця:  
[[0.6 0.6 0.6]  
 [0.8 0.8 0.8]]  
Норми: [ 5. 10. 15.]
```

Для кожного стовпця матриці ознак обчислюється норма L2 за допомогою функції `'np.linalg.norm()'`. Після цього кожен елемент стовпця ділиться на відповідну норму, внаслідок чого всі ознаки набувають однакового масштабу. Функція повертає нормалізовану матрицю ознак та масив норм, які можуть бути використані для подальшого відновлення початкового масштабу коефіцієнтів.

2.2 Один крок координатного спуску

Реалізуйте `lasso_coordinate_descent_step(i, feature_matrix, output, weights, l1_penalty)`. Функція має:

- обчислити поточне передбачення;
- обчислити ρ_i ;
- якщо $i = 0$: $w_i = \rho_i$ (без штрафу). Якщо $i \geq 1$: застосувати правило м'якого порогоування;
- повернути нове значення ваги.

```
[7]
✓ Os ● def lasso_coordinate_descent_step(i, feature_matrix, output, weights, l1_penalty):
    """
    Виконує один крок координатного спуску LASSO для ваги з індексом i.

    Параметри
    -----
    i          : int      індекс ваги для оновлення
    feature_matrix: array  нормалізована, shape (n, d)
    output      : array   shape (n,)
    weights     : array   поточні ваги, shape (d,)
    l1_penalty  : float   λ

    Повертає
    -----
    new_weight_i : float
    """
    prediction = predict_output(feature_matrix, weights)

    # Compute rho_i
    rho_i = np.dot(
        feature_matrix[:, i],
        output - (prediction - weights[i] * feature_matrix[:, i])
    )

    # Apply soft-threshold (intercept is unregularised)
    if i == 0: # intercept – no penalty
        new_weight_i = rho_i
    else:
        if rho_i < -l1_penalty / 2:
            new_weight_i = rho_i + l1_penalty / 2
        elif rho_i > l1_penalty / 2:
            new_weight_i = rho_i - l1_penalty / 2
        else:
            new_weight_i = 0.0

    return new_weight_i

▼ Перевірка — результат має бути ≈ 0.4256

[8]
✓ Os ● from math import sqrt
    result = lasso_coordinate_descent_step(
        i=1,
        feature_matrix=np.array([[3/sqrt(13), 1/sqrt(10)],
                                [2/sqrt(13), 3/sqrt(10)]]),
        output=np.array([1., 1.]),
        weights=np.array([1., 4.]),
        l1_penalty=0.1
    )
    print(f'Результат кроку: {result:.4f} (очікується ≈ 0.4256)')

... Результат кроку: 0.4256 (очікується ≈ 0.4256)
```

Спочатку обчислюються поточні передбачення моделі. Далі для вибраної ознаки визначається значення p_i , яке використовується для оновлення відповідної ваги. Для вільного члена ($i = 0$) нова вага прирівнюється до p_i без застосування регуляризації. Для всіх інших ваг використовується правило м'якого порогування, яке зменшує значення коефіцієнтів і може занулювати їх при достатньо великому значенні параметра регуляризації `l1_penalty`.

2.3 Циклічний координатний спуск

Реалізуємо `lasso_cyclical_coordinate_descent(feature_matrix, output, initial_weights, l1_penalty, tolerance)`. У кожному циклі оновлюйте ваги 0, 1, ..., $d-1$ по черзі, використовуючи оновлену вагу одразу. Зупинемося, коли максимальна абсолютна зміна ваги за цикл $< \text{tolerance}$.

```
[9]
✓ Os def lasso_cyclical_coordinate_descent(feature_matrix, output,
                                         initial_weights, l1_penalty, tolerance):
    """
    Запускає циклічний координатний спуск до збіжності.

    Повертає
    -----
    weights : np.ndarray — навчені ваги (на нормалізованих ознаках)
    """
    weights = np.array(initial_weights, dtype=float)
    converged = False

    while not converged:
        max_change = 0.0
        for i in range(len(weights)):
            old_weight_i = weights[i]
            # Update weight i
            weights[i] = lasso_coordinate_descent_step(
                i,
                feature_matrix,
                output,
                weights,
                l1_penalty
            )
            # Track the largest change this cycle
            max_change = max(max_change, abs(weights[i] - old_weight_i))

        if max_change < tolerance:
            converged = True

    return weights
```

Реалізується циклічний координатний спуск, у якому ваги моделі оновлюються послідовно одна за одною. Для кожної ваги викликається функція `lasso_coordinate_descent_step()`, а отримане нове значення одразу

використовується під час оновлення наступних ваг. Після завершення кожного циклу визначається найбільша зміна серед усіх ваг. Алгоритм завершує роботу, коли максимальна зміна ваг стає меншою за заданий поріг `tolerance`.

2.4 Відбір ознак

```
tolerance = 1.0

- l1_penalty = 1e7: вкажіть навчені ваги, RSS та які ознаки мають ненульові ваги.
- l1_penalty = 1e8: повторіть. Що змінилося?

[10]
✓ Os
sales = pd.read_csv('kc_house_data.csv')

# Побудуйте та нормалізуйте матрицю ознак
feature_matrix, output = get_numpy_data(sales, ['sqft_living', 'bedrooms'], 'price')
normalised_features, feature_norms = normalize_features(feature_matrix)

[11]
✓ Os
# Запуск з l1_penalty = 1e7
weights_1e7 = lasso_cyclical_coordinate_descent(
    normalised_features, output,
    initial_weights=np.zeros(3),
    l1_penalty=1e7,
    tolerance=1.0
)
print('Ваги (λ=1e7):', weights_1e7)
print('Non-zero features:', ['intercept', 'sqft_living', 'bedrooms'][:
    len([w for w in weights_1e7 if w != 0])]) # rough indicator

# RSS on the normalised dataset
preds_1e7 = predict_output(normalised_features, weights_1e7)
rss_1e7 = np.sum((preds_1e7 - output) ** 2)
print(f'RSS (λ=1e7): {rss_1e7:.3e}')

Ваги (λ=1e7): [21624997.95951911 63157247.20788956      0.      ]
Non-zero features: ['intercept', 'sqft_living']
RSS (λ=1e7): 1.630e+15

[12]
✓ Os
# Запуск з l1_penalty = 1e8
weights_1e8 = lasso_cyclical_coordinate_descent(
    normalised_features, output,
    initial_weights=np.zeros(3),
    l1_penalty=1e8,
    tolerance=1.0
)
print('Ваги (λ=1e8):', weights_1e8)

... Ваги (λ=1e8): [79400304.63764459      0.      0.      ]

Спостереження: При значенні штрафу  $\lambda = 1e7$  ненульовими залишилися вільний член та ознака sqft_living, тоді як вага ознаки bedrooms була занулена. При збільшенні штрафу до  $\lambda = 1e8$  регуляризація стала сильнішою, тому було занулено також вагу ознаки sqft_living, і в моделі залишився лише вільний член. Таким чином, зі збільшенням параметра регуляризації LASSO відкидає більше ознак, спрощуючи модель та виконуючи автоматичний відбір найбільш важливих змінних.
```

2.5 Бонус — Оцінка на ненормалізованих тестових даних

Розіб'ємо дані 80 % / 20 % (`random_state=42`). Побудуємо та нормалізуємо навчальну матрицю ознак з усіма 10 ознаками нижче. Запустимо координатний спуск (`l1_penalty=1e7, tolerance=1.0`).

Ознаки: sqft_living, bedrooms, bathrooms, sqft_lot, floors, waterfront, view, condition, grade, yr_built.

Щоб оцінювати на ненормалізованих тестових даних, масштабуємо ваги:
$$\text{weights_rescaled} = \text{weights} / \text{norms}$$

Обчислимо RSS на ненормалізованій тестовій матриці ознак. Вкажемо відібрані ознаки.

```
Обчисліть RSS на ненормалізованій тестовій матриці ознак. Вкажіть відібрані ознаки.

[13] ✓ Os ● FEATURES = [
    'sqft_living', 'bedrooms', 'bathrooms', 'sqft_lot',
    'floors', 'waterfront', 'view', 'condition',
    'grade', 'yr_built'
]

train_data, test_data = train_test_split(sales, test_size=0.2, random_state=42)

# Бонус – нормалізуйте навчальні ознаки, запустіть LASSO, масштабуйте ваги, оцініть
# Навчальна матриця ознак
train_feature_matrix, train_output = get_numpy_data(
    train_data,
    FEATURES,
    'price'
)

# Нормалізація
normalised_train_features, norms = normalize_features(
    train_feature_matrix
)

# Навчання LASSO
weights = lasso_cyclical_coordinate_descent(
    normalised_train_features,
    train_output,
    initial_weights=np.zeros(len(FEATURES) + 1),
    l1_penalty=1e7,
    tolerance=1.0
)

# Масштабування ваг для роботи з ненормалізованими даними
weights_rescaled = weights / norms

# Тестова матриця без нормалізації
test_feature_matrix, test_output = get_numpy_data(
    test_data,
    FEATURES,
    'price'
)

# Передбачення та RSS
test_predictions = predict_output(
    test_feature_matrix,
    weights_rescaled
)

rss_test = np.sum((test_output - test_predictions) ** 2)

print('Масштабовані ваги:')
print(weights_rescaled)

print(f'\nRSS на тестовій вибірці: {rss_test:.3e}')

selected_features = ['intercept'] + FEATURES
```



```
selected_features = ['intercept'] + FEATURES

print('\nВідібрані ознаки:')
for feature, weight in zip(selected_features, weights):
    if weight != 0:
        print(feature)
```

```
*** Масштабовані ваги:
[1.95381239e+05 1.56107371e+02 0.00000000e+00 0.00000000e+00
 0.00000000e+00 0.00000000e+00 2.11864433e+05 7.34560652e+04
 0.00000000e+00 0.00000000e+00 0.00000000e+00]
```

RSS на тестовій вибірці: 3.425e+14

Відібрані ознаки:
intercept
sqft_living
waterfront
view

Відібрані ознаки: *intercept, sqft_living, waterfront та view*

Тестова RSS: 3.425e+14

Інтерпретація: Вибрані ознаки мають інтуїтивний сенс для прогнозування вартості будинку. Житлова площа (*sqft_living*) безпосередньо впливає на ціну нерухомості, а наявність виходу до води (*waterfront*) та привабливий вигляд (*view*) є важливими факторами, які суттєво підвищують ринкову вартість будинку. Тому відбір саме цих ознак можна вважати обґрунтованим.

3. Висновки: у ході роботи було реалізовано нормалізацію ознак, один крок та повний алгоритм циклічного координатного спуску для LASSO-регресії без використання готових реалізацій scikit-learn. Досліджено вплив параметра регуляризації на відбір ознак та встановлено, що зі збільшенням штрафу кількість ненульових коефіцієнтів зменшується, що призводить до спрощення моделі. Для набору з десяти ознак алгоритм відібрав `sqft\_living`, `waterfront` і `view`, які мають логічний вплив на вартість нерухомості, а отримане значення тестової RSS становило 3.425×10^{14} .

4. Покликання

Репозиторій GitHub: [покликання](#)

Відео на YouTube: [покликання](#)